НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Звіт до комп'ютерного практикуму №6
З дисципліни «Комп'ютерне моделювання дискретно-подійних систем»
«Застосування алгоритму стохастичної мережі Петрі для реалізації моделей дискретно-подійних систем»

Прийняв:
Викладач
Коринецький Дмитро Сергійович
«12» листопада 2026 р.

Виконав:
Студент 4 курсу, гр. ІП-34
Скачков Іван
Олексійович

2026 р.

Зміст

Лабораторна робота №6 3
Технічне завдання 4
Виконання 5
Бібліотека PetriObjModelPaint 5
Лістинг допоміжного класу 6
Завдання 1 8
Лістинг файлів 8
Результат роботи 10
Висновок 11
Завдання 2 12
Лістинг файлів 12
Результат роботи 14
Висновок 15
Завдання 3 16
Формалізація 16
Лістинг файлів 17
Результат роботи 18
Висновок 18
Завдання 4 20
Завдання 5 21
Висновок 23
Посилання 24

Лабораторна робота №6

Тема: Застосування алгоритму стохастичної мережі Петрі для реалізації моделей дискретно-подійних систем.

Мета: ознайомитися з бібліотекою PetriObjModelPaint і її графічним редактором, реалізувати моделі дискретно-подійних систем алгоритмом імітації стохастичної мережі Петрі класу PetriSim, виконати їх верифікацію та описати мережу Петрі математичними рівняннями.

Засоби: PetriObjModelPaint 2.8.2 (І. В. Стеценко): графічний редактор мереж Петрі і бібліотека petri-math; Java 25.

Технічне завдання

1. Ознайомитися з бібліотекою класів PetriObjModelPaint моделювання дискретно-подійних систем на основі стохастичних мереж Петрі та графічним редактором мережі Петрі. З використанням алгоритму імітації стохастичної мережі Петрі класу PetriSim реалізувати модель, розроблену за текстом завдання 1 практикуму 5, та виконати її верифікацію. Зробити висновки про функціонування моделі.
2. З використанням алгоритму імітації стохастичної мережі Петрі класу PetriSim реалізувати модель, розроблену за текстом завдання 4 практикуму 5, та виконати її верифікацію. Зробити висновки про функціонування моделі.
3. Побудувати модель системи, що відтворює багатопотокову обробку запитів на кожному з двох серверів: головному та допоміжному. Для запобігання DDOS-атакам для черги запитів встановлено обмеження до 1000. Результат обробки запиту обов’язково зберігається в базі даних перед його поверненням користувачу. База даних є спільною для обох серверів і забезпечена багатопотоковим доступом. Ймовірність звернення до допоміжного сервера 0,3. Часові характеристики обробки запитів, кількість обчислювального ресурсу та кількість потоків задати самостійно. Метою моделювання є оцінка часу обробки запиту.
4. Для усіх розроблених мереж Петрі створити та подати разом зі звітом пакет файлів: 1) текст завдання, 2) зображення мережі Петрі у форматі .png, 3) результат збереження мережі Петрі у форматі .pnml, 4) результат збереження мережі Петрі у вигляді Java-методу у форматі .txt.
5. Побудувати математичні рівняння, що описують побудовану за текстом завдання 3 мережу Петрі.

Виконання

Бібліотека PetriObjModelPaint

Бібліотека реалізує техніку Петрі-об’єктного моделювання: поведінка кожного об’єкта моделі описується стохастичною багатоканальною мережею Петрі, а модель складається з об’єктів і зв’язків між ними. Основні класи наведено в таблиці 1.

Таблиця 1 – Основні класи бібліотеки

Клас	Призначення
PetriP	Позиція: назва, кількість маркерів, середня за часом кількість маркерів
PetriT	Перехід: часова затримка і закон її розподілу (exp, unif, norm або детермінована), пріоритет, ймовірність запуску, список моментів виходу маркерів (багатоканальність), середня кількість маркерів у переході
ArcIn, ArcOut	Вхідна і вихідна дуги з кратністю; вхідна дуга може бути інформаційною (setInf(true))
PetriNet	Мережа Петрі: списки позицій, переходів і дуг, перевірка коректності структури
PetriSim	Петрі-об’єкт і алгоритм імітації стохастичної мережі Петрі
PetriObjModel	Модель зі списку Петрі-об’єктів: просування часу і збирання статистики
FunRand	Генератори випадкових величин

Алгоритм класу PetriSim відповідає алгоритму імітації стохастичної мережі Петрі з лекції 10. Спочатку виконується вхід маркерів: формується список переходів, умова запуску яких виконана (у вхідних позиціях достатньо маркерів, кількість яких не менша за кратність дуги), список сортується за пріоритетом, конфлікт розв’язується на користь переходу з найбільшим пріоритетом, а серед рівних за пріоритетом — випадково з урахуванням імовірностей запуску. Обраний перехід забирає маркери з вхідних позицій (крім інформаційних дуг) і додає в свій список момент виходу $t + \tau$. Вхід повторюється, доки є переходи з виконаною умовою, тому багатоканальний перехід може запуститися кілька разів. Потім час просувається до найближчого моменту виходу, і маркери виходять з усіх переходів, момент виходу яких настав. Після цього знову виконується вхід — і так до кінця моделювання.

Робота з бібліотекою організована так. Мережу будують у графічному редакторі, перевіряють запуском з анімацією і зберігають командою Save net as method: редактор генерує Java-метод, що створює об’єкт PetriNet. Методи зібрано в класі Nets.java, де параметри експериментів винесено в аргументи. Допоміжний клас Sim.java загортає мережу в PetriSim, запускає модель PetriObjModel на заданий час і дає доступ до позицій і переходів за назвою.

Під час налагодження виявлено особливість розв’язання конфлікту: у розіграші беруть участь усі активні переходи з тим самим пріоритетом, навіть якщо вони не мають спільних вхідних позицій, а перехід з імовірністю 1,0 отримує частку 1/k. Перехід-генератор стає активним у той самий момент, коли з’являється нова вимога, тож без додаткових заходів він спотворював задані ймовірності: замість 0,8 і 0,2 виходило 0,93 і 0,07. Тому переходи вибору за ймовірністю отримали пріоритет 1, вищий за пріоритет генератора.

Лістинг допоміжного класу

Файл src\Sim.java:

```
import java.util.ArrayList;
import java.util.List;
import ua.stetsenkoinna.petriobj.PetriNet;
import ua.stetsenkoinna.petriobj.PetriObjModel;
import ua.stetsenkoinna.petriobj.PetriP;
import ua.stetsenkoinna.petriobj.PetriSim;
import ua.stetsenkoinna.petriobj.PetriT;
import ua.stetsenkoinna.petriobj.StateTime;


/**
 * Імітація мережі Петрі: Петрі-об'єкт PetriSim у моделі PetriObjModel, яка просуває час
 * до найближчої події і рахує середні за часом значення маркерів і зайнятості переходів.
 */
final class Sim {
    final PetriNet net;
    double time;


    Sim(PetriNet net) {
        this.net = net;
    }


    /** Запам'ятовувати моменти входу і виходу маркерів переходу (щоб рахувати запуски). */
    Sim track(String... transitions) {
        for (String name : transitions) {
            t(name).setMoments(true);
        }
        return this;
    }


    Sim run(double time) {
        this.time = time;
        ArrayList<PetriSim> objects = new ArrayList<>(List.of(new PetriSim(net, new StateTime())));
        PetriObjModel model = new PetriObjModel(objects);
        model.setIsProtokol(false);
```

```
        model.go(time);
        return this;
    }


    PetriP p(String name) {
        for (PetriP place : net.getListP()) {
            if (place.getName().equals(name)) {
                return place;
            }
        }
        throw new IllegalArgumentException("немає позиції " + name);
    }


    PetriT t(String name) {
        for (PetriT transition : net.getListT()) {
            if (transition.getName().equals(name)) {
                return transition;
            }
        }
        throw new IllegalArgumentException("немає переходу " + name);
    }


    /** Кількість маркерів у позиції в кінці моделювання. */
    int mark(String place) {
        return p(place).getMark();
    }


    /** Середня за часом кількість маркерів у позиції. */
    double mean(String place) {
        return p(place).getMean();
    }


    /** Середня за часом кількість маркерів, що одночасно перебувають у переході. */
    double busy(String transition) {
        return t(transition).getMean();
    }


    /** Скільки разів маркери вийшли з переходу (потрібен track). */
    int fired(String transition) {
        return t(transition).getOutMoments().size();
    }


    /** Моменти виходу маркерів з переходу (потрібен track). */
    List<Double> exits(String transition) {
        return t(transition).getOutMoments();
    }
}
```

Завдання 1

Мережу конвеєрної системи, розроблену в практикумі 5, наведено на рисунку 1: деталь біля пристрою і обробляється (перехід «обробка і», пріоритет 1), якщо пристрій вільний, і переїжджає до наступного пристрою (перехід «конвеєр», пріоритет 0, затримка 1 хв), якщо зайнятий; від п’ятого пристрою деталь повертається до першого із затримкою 5 хв. Мережа реалізована методом CreateNetConveyor, отриманим командою Save net as method; інтервал надходження деталей винесено в параметр.

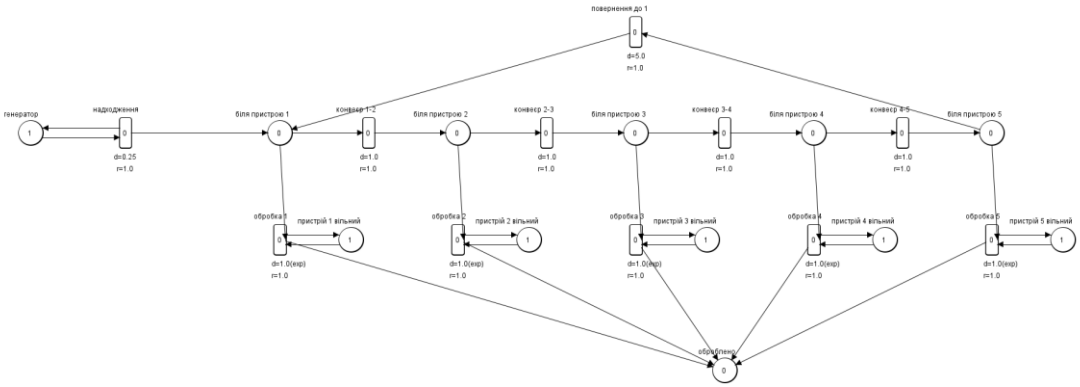


Рисунок 1 – Мережа Петрі конвеєрної системи

Лістинг файлів

Файл nets\1_conveyor\method.txt:

```
public static PetriNet CreateNetConveyor() throws ExceptionInvalidNetStructure, ExceptionInvalidTimeDelay {
        ArrayList<PetriP> d_P = new ArrayList<>();
        ArrayList<PetriT> d_T = new ArrayList<>();
        ArrayList<ArcIn> d_In = new ArrayList<>();
        ArrayList<ArcOut> d_Out = new ArrayList<>();
        d_P.add(new PetriP("генератор",1));
        d_P.add(new PetriP("біля пристрою 1",0));
        d_P.add(new PetriP("пристрій 1 вільний",1));
        d_P.add(new PetriP("біля пристрою 2",0));
        d_P.add(new PetriP("пристрій 2 вільний",1));
        d_P.add(new PetriP("біля пристрою 3",0));
        d_P.add(new PetriP("пристрій 3 вільний",1));
        d_P.add(new PetriP("біля пристрою 4",0));
        d_P.add(new PetriP("пристрій 4 вільний",1));
        d_P.add(new PetriP("біля пристрою 5",0));
        d_P.add(new PetriP("пристрій 5 вільний",1));
        d_P.add(new PetriP("оброблено",0));
        d_T.add(new PetriT("надходження",0.25));
        d_T.add(new PetriT("обробка 1",1.0));
        d_T.get(1).setDistribution("exp", d_T.get(1).getTimeServ());
        d_T.get(1).setParamDeviation(0.0);
        d_T.get(1).setPriority(1);
        d_T.add(new PetriT("конвеєр 1-2",1.0));
        d_T.add(new PetriT("обробка 2",1.0));
        d_T.get(3).setDistribution("exp", d_T.get(3).getTimeServ());
        d_T.get(3).setParamDeviation(0.0);
        d_T.get(3).setPriority(1);
```

```
        d_T.add(new PetriT("конвеєр 2-3",1.0));
        d_T.add(new PetriT("обробка 3",1.0));
        d_T.get(5).setDistribution("exp", d_T.get(5).getTimeServ());
        d_T.get(5).setParamDeviation(0.0);
        d_T.get(5).setPriority(1);
        d_T.add(new PetriT("конвеєр 3-4",1.0));
        d_T.add(new PetriT("обробка 4",1.0));
        d_T.get(7).setDistribution("exp", d_T.get(7).getTimeServ());
        d_T.get(7).setParamDeviation(0.0);
        d_T.get(7).setPriority(1);
        d_T.add(new PetriT("конвеєр 4-5",1.0));
        d_T.add(new PetriT("обробка 5",1.0));
        d_T.get(9).setDistribution("exp", d_T.get(9).getTimeServ());
        d_T.get(9).setParamDeviation(0.0);
        d_T.get(9).setPriority(1);
        d_T.add(new PetriT("повернення до 1",5.0));
        d_In.add(new ArcIn(d_P.get(0),d_T.get(0),1));
        d_In.add(new ArcIn(d_P.get(1),d_T.get(1),1));
        d_In.add(new ArcIn(d_P.get(2),d_T.get(1),1));
        d_In.add(new ArcIn(d_P.get(1),d_T.get(2),1));
        d_In.add(new ArcIn(d_P.get(3),d_T.get(3),1));
        d_In.add(new ArcIn(d_P.get(4),d_T.get(3),1));
        d_In.add(new ArcIn(d_P.get(3),d_T.get(4),1));
        d_In.add(new ArcIn(d_P.get(5),d_T.get(5),1));
        d_In.add(new ArcIn(d_P.get(6),d_T.get(5),1));
        d_In.add(new ArcIn(d_P.get(5),d_T.get(6),1));
        d_In.add(new ArcIn(d_P.get(7),d_T.get(7),1));
        d_In.add(new ArcIn(d_P.get(8),d_T.get(7),1));
        d_In.add(new ArcIn(d_P.get(7),d_T.get(8),1));
        d_In.add(new ArcIn(d_P.get(9),d_T.get(9),1));
        d_In.add(new ArcIn(d_P.get(10),d_T.get(9),1));
        d_In.add(new ArcIn(d_P.get(9),d_T.get(10),1));
        d_Out.add(new ArcOut(d_T.get(0),d_P.get(0),1));
        d_Out.add(new ArcOut(d_T.get(0),d_P.get(1),1));
        d_Out.add(new ArcOut(d_T.get(1),d_P.get(2),1));
        d_Out.add(new ArcOut(d_T.get(1),d_P.get(11),1));
        d_Out.add(new ArcOut(d_T.get(2),d_P.get(3),1));
        d_Out.add(new ArcOut(d_T.get(3),d_P.get(4),1));
        d_Out.add(new ArcOut(d_T.get(3),d_P.get(11),1));
        d_Out.add(new ArcOut(d_T.get(4),d_P.get(5),1));
        d_Out.add(new ArcOut(d_T.get(5),d_P.get(6),1));
        d_Out.add(new ArcOut(d_T.get(5),d_P.get(11),1));
        d_Out.add(new ArcOut(d_T.get(6),d_P.get(7),1));
        d_Out.add(new ArcOut(d_T.get(7),d_P.get(8),1));
        d_Out.add(new ArcOut(d_T.get(7),d_P.get(11),1));
        d_Out.add(new ArcOut(d_T.get(8),d_P.get(9),1));
        d_Out.add(new ArcOut(d_T.get(9),d_P.get(10),1));
        d_Out.add(new ArcOut(d_T.get(9),d_P.get(11),1));
        d_Out.add(new ArcOut(d_T.get(10),d_P.get(1),1));
        PetriNet d_Net = new PetriNet("Conveyor",d_P,d_T,d_In,d_Out);
        PetriP.initNext();
        PetriT.initNext();
        ArcIn.initNext();
        ArcOut.initNext();


        return d_Net;
}
```

Файл src\Task1Conveyor.java:

```
/** Завдання 1: модель конвеєра (завдання 1 практикуму 5) на основі PetriSim і її верифікація. */
public class Task1Conveyor {
    static final double TIME = 10_000, SERVICE = 1, STEP = 1, BACK = 5;
    static final String[] MOVES = {"конвеєр 1-2", "конвеєр 2-3", "конвеєр 3-4", "конвеєр 4-5"};


    record Result(int came, int done, double[] loads, double busy, double inSystem,
                  double timeLittle, double timePaths, double passes, double returns) {
    }


    static Result run(double interval) throws Exception {
        Sim s = new Sim(Nets.CreateNetConveyor(interval))
                .track("надходження", "повернення до 1").track(MOVES).run(TIME);
        double[] loads = new double[5];
        double busy = 0, inSystem = 0;
        for (int i = 1; i <= 5; i++) {
            loads[i - 1] = 1 - s.mean("пристрій " + i + " вільний");
            busy += loads[i - 1];
            inSystem += s.busy("обробка " + i);
        }
        int passes = 0;
        for (String move : MOVES) {
```

```
            passes += s.fired(move);
            inSystem += s.busy(move);
        }
        inSystem += s.busy("повернення до 1");
        int done = s.mark("оброблено"), returns = s.fired("повернення до 1");
        double p = (double) passes / done, r = (double) returns / done;
        return new Result(s.fired("надходження"), done, loads, busy, inSystem,
                inSystem / (done / TIME), SERVICE + STEP * p + BACK * r, p, r);
    }


    public static void main(String[] args) throws Exception {
        Result r = run(0.25);
        System.out.printf("Конвеєр, T = %.0f хв, 4 деталі/хв: надійшло %d, оброблено %d%n",
                TIME, r.came, r.done);
        for (int i = 0; i < 5; i++) {
            System.out.printf("  завантаження пристрою %d: %.4f%n", i + 1, r.loads[i]);
        }
        System.out.printf("Середня кількість зайнятих пристроїв: %.4f%n", r.busy);
        System.out.printf("Середня кількість деталей у системі: %.4f%n", r.inSystem);
        System.out.printf("Середній час перебування: %.4f хв (Літтл), "
                + "%.4f хв (обробка + переміщення)%n", r.timeLittle, r.timePaths);


        System.out.println("\nВерифікація: зміна інтенсивності надходження деталей");
        System.out.println(" λ, дет/хв | зайнято пристроїв | λ·тобр | завантаження 1..5"
                + "              | переміщень | повернень | час, хв");
        for (double rate : new double[]{1, 2, 3, 4, 4.5}) {
            Result v = run(1 / rate);
            StringBuilder loads = new StringBuilder();
            for (double l : v.loads) {
                loads.append(String.format("%.3f ", l));
            }
            System.out.printf(" %9.1f | %17.4f | %6.2f | %s| %10.3f | %9.3f | %7.3f%n", rate,
                    v.busy, rate * SERVICE, loads, v.passes, v.returns, v.timeLittle);
        }
    }
}
```

Результат роботи

```
Конвеєр, T = 10000 хв, 4 деталі/хв: надійшло 40000, оброблено 39961
  завантаження пристрою 1: 0.8847
  завантаження пристрою 2: 0.8560
  завантаження пристрою 3: 0.8204
  завантаження пристрою 4: 0.7639
  завантаження пристрою 5: 0.6993
Середня кількість зайнятих пристроїв: 4.0244
Середня кількість деталей у системі: 39.5826
Середній час перебування: 9.9053 хв (Літтл), 9.8966 хв (обробка + переміщення)


Верифікація: зміна інтенсивності надходження деталей
 λ, дет/хв | зайнято пристроїв | λ·тобр | завантаження 1..5              | переміщень | повернень | час, хв
       1.0 |            0.9994 |   1.00 | 0.635 0.301 0.059 0.004 0.000 |      0.442 |     0.000 |   1.441
       2.0 |            2.0040 |   2.00 | 0.784 0.624 0.396 0.163 0.036 |      1.021 |     0.002 |   2.035
       3.0 |            3.0072 |   3.00 | 0.852 0.770 0.642 0.473 0.270 |      1.762 |     0.061 |   3.068
       4.0 |            3.9987 |   4.00 | 0.885 0.855 0.810 0.760 0.689 |      5.050 |     0.793 |  10.017
       4.5 |            4.4536 |   4.50 | 0.927 0.912 0.895 0.873 0.847 |     10.585 |     2.155 |  22.354
```

Таблиця 2 – Верифікація моделі конвеєра: зміна інтенсивності надходження λ

λ, дет./хв	**Зайнято пристроїв**	**λ·тобр**	**Завантаження пристроїв 1…5**	**Переміщень на деталь**	**Повернень на деталь**
1,0	0,999	1,0	0,64 / 0,30 / 0,06 / 0,00 / 0,00	0,44	0,00
2,0	2,004	2,0	0,78 / 0,62 / 0,40 / 0,16 / 0,04	1,02	0,00
3,0	3,007	3,0	0,85 / 0,77 / 0,64 / 0,47 / 0,27	1,76	0,06
4,0	3,999	4,0	0,89 / 0,85 / 0,81 / 0,76 / 0,69	5,05	0,79
4,5	4,454	4,5	0,93 / 0,91 / 0,90 / 0,87 / 0,85	10,59	2,15

Висновок

Модель відтворює роботу конвеєра: при надходженні 4 деталей/хв завантаження пристроїв спадає від 0,88 до 0,70, у середньому зайнято 4,02 пристрою, а деталь перебуває в системі 9,91 хв. Цей час, обчислений за формулою Літтла, збігається з часом, отриманим із кількості обробок і переміщень (9,90 хв).

Верифікація (таблиця 2) підтверджує правильність моделі. Середня кількість зайнятих пристроїв у всіх прогонах дорівнює добутку інтенсивності надходження на середній час обробки, бо кожна деталь обробляється рівно один раз. При малій інтенсивності деталі обробляються переважно на першому пристрої і майже не повертаються; зі збільшенням інтенсивності завантаження вирівнюється, а кількість переміщень і повернень швидко зростає. При λ, що наближається до продуктивності п’яти пристроїв (5 деталей/хв), деталь багато разів проходить конвеєр, і час перебування в системі різко збільшується.

Завдання 2

Мережу системи управління запасами холодильників (практикум 5) наведено на рисунку 2. Поточний стан запасу подано через позицію «нестача до рівня», кількість маркерів у якій дорівнює контрольному рівню мінус поточний стан запасу; умову «стан запасу ≤ точки замовлення» перевіряє інформаційна дуга з кратністю «рівень − точка» (54). Метод CreateNetFridges має параметри: контрольний рівень (він же початковий запас), точку замовлення і період перегляду.

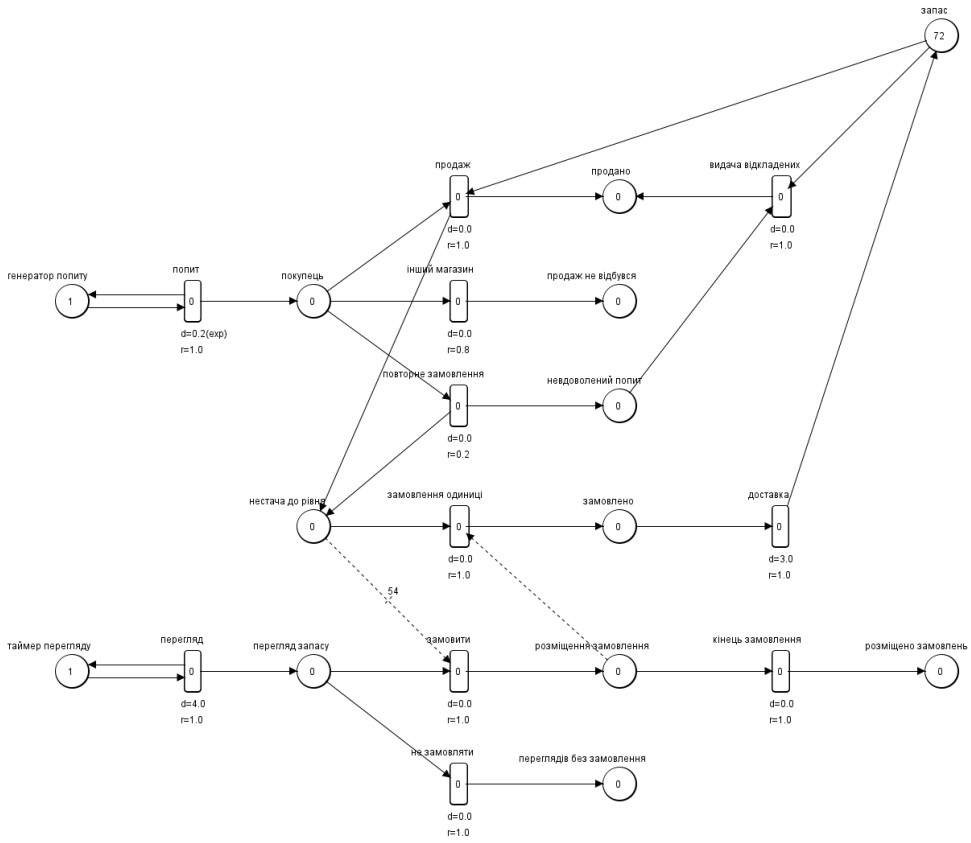


Рисунок 2 – Мережа Петрі системи управління запасами холодильників

Лістинг файлів

Файл nets\2_fridges\method.txt:

```
public static PetriNet CreateNetFridges() throws ExceptionInvalidNetStructure, ExceptionInvalidTimeDelay {
        ArrayList<PetriP> d_P = new ArrayList<>();
        ArrayList<PetriT> d_T = new ArrayList<>();
        ArrayList<ArcIn> d_In = new ArrayList<>();
        ArrayList<ArcOut> d_Out = new ArrayList<>();
        d_P.add(new PetriP("генератор попиту",1));
        d_P.add(new PetriP("покупець",0));
```

```
        d_P.add(new PetriP("запас",72));
        d_P.add(new PetriP("продано",0));
        d_P.add(new PetriP("продаж не відбувся",0));
        d_P.add(new PetriP("невдоволений попит",0));
        d_P.add(new PetriP("нестача до рівня",0));
        d_P.add(new PetriP("таймер перегляду",1));
        d_P.add(new PetriP("перегляд запасу",0));
        d_P.add(new PetriP("розміщення замовлення",0));
        d_P.add(new PetriP("переглядів без замовлення",0));
        d_P.add(new PetriP("замовлено",0));
        d_P.add(new PetriP("розміщено замовлень",0));
        d_T.add(new PetriT("попит",0.2));
        d_T.get(0).setDistribution("exp", d_T.get(0).getTimeServ());
        d_T.get(0).setParamDeviation(0.0);
        d_T.add(new PetriT("продаж",0.0));
        d_T.get(1).setPriority(2);
        d_T.add(new PetriT("інший магазин",0.0));
        d_T.get(2).setPriority(1);
        d_T.get(2).setProbability(0.8);
        d_T.add(new PetriT("повторне замовлення",0.0));
        d_T.get(3).setPriority(1);
        d_T.get(3).setProbability(0.2);
        d_T.add(new PetriT("видача відкладених",0.0));
        d_T.get(4).setPriority(3);
        d_T.add(new PetriT("перегляд",4.0));
        d_T.add(new PetriT("замовити",0.0));
        d_T.get(6).setPriority(2);
        d_T.add(new PetriT("не замовляти",0.0));
        d_T.get(7).setPriority(1);
        d_T.add(new PetriT("замовлення одиниці",0.0));
        d_T.get(8).setPriority(3);
        d_T.add(new PetriT("доставка",3.0));
        d_T.add(new PetriT("кінець замовлення",0.0));
        d_T.get(10).setPriority(1);
        d_In.add(new ArcIn(d_P.get(0),d_T.get(0),1));
        d_In.add(new ArcIn(d_P.get(1),d_T.get(1),1));
        d_In.add(new ArcIn(d_P.get(2),d_T.get(1),1));
        d_In.add(new ArcIn(d_P.get(1),d_T.get(2),1));
        d_In.add(new ArcIn(d_P.get(1),d_T.get(3),1));
        d_In.add(new ArcIn(d_P.get(5),d_T.get(4),1));
        d_In.add(new ArcIn(d_P.get(2),d_T.get(4),1));
        d_In.add(new ArcIn(d_P.get(7),d_T.get(5),1));
        d_In.add(new ArcIn(d_P.get(8),d_T.get(6),1));
        d_In.add(new ArcIn(d_P.get(6),d_T.get(6),54));
        d_In.get(9).setInf(true);
        d_In.add(new ArcIn(d_P.get(8),d_T.get(7),1));
        d_In.add(new ArcIn(d_P.get(6),d_T.get(8),1));
        d_In.add(new ArcIn(d_P.get(9),d_T.get(8),1));
        d_In.get(12).setInf(true);
        d_In.add(new ArcIn(d_P.get(11),d_T.get(9),1));
        d_In.add(new ArcIn(d_P.get(9),d_T.get(10),1));
        d_Out.add(new ArcOut(d_T.get(0),d_P.get(0),1));
        d_Out.add(new ArcOut(d_T.get(0),d_P.get(1),1));
        d_Out.add(new ArcOut(d_T.get(1),d_P.get(3),1));
        d_Out.add(new ArcOut(d_T.get(1),d_P.get(6),1));
        d_Out.add(new ArcOut(d_T.get(2),d_P.get(4),1));
        d_Out.add(new ArcOut(d_T.get(3),d_P.get(5),1));
        d_Out.add(new ArcOut(d_T.get(3),d_P.get(6),1));
        d_Out.add(new ArcOut(d_T.get(4),d_P.get(3),1));
        d_Out.add(new ArcOut(d_T.get(5),d_P.get(7),1));
        d_Out.add(new ArcOut(d_T.get(5),d_P.get(8),1));
        d_Out.add(new ArcOut(d_T.get(6),d_P.get(9),1));
        d_Out.add(new ArcOut(d_T.get(7),d_P.get(10),1));
        d_Out.add(new ArcOut(d_T.get(8),d_P.get(11),1));
        d_Out.add(new ArcOut(d_T.get(9),d_P.get(2),1));
        d_Out.add(new ArcOut(d_T.get(10),d_P.get(12),1));
        PetriNet d_Net = new PetriNet("Fridges",d_P,d_T,d_In,d_Out);
        PetriP.initNext();
        PetriT.initNext();
        ArcIn.initNext();
        ArcOut.initNext();


        return d_Net;
}
```

Файл src\Task2Fridges.java:

```
import java.util.List;


/** Завдання 2: модель запасів (завдання 4 практикуму 5) на основі PetriSim і її верифікація. */
public class Task2Fridges {
```

```
    static final double TIME = 20_000;


    record Result(int customers, int sold, int lost, int backlog, double stock, double between,
                  double lostShare, int orders, double orderSize) {
    }


    static Result run(int level, int reorder, double review) throws Exception {
        Sim s = new Sim(Nets.CreateNetFridges(level, reorder, review))
                .track("попит", "інший магазин", "доставка").run(TIME);
        List<Double> lostAt = s.exits("інший магазин");
        int customers = s.fired("попит"), lost = s.mark("продаж не відбувся");
        int orders = s.mark("розміщено замовлень");
        double between = (lostAt.getLast() - lostAt.getFirst()) / (lostAt.size() - 1);
        return new Result(customers, s.mark("продано"), lost, s.mark("невдоволений попит"),
                s.mean("запас"), between, (double) lost / customers, orders,
                (double) s.fired("доставка") / orders);
    }


    public static void main(String[] args) throws Exception {
        Result r = run(72, 18, 4);
        System.out.printf("Запаси холодильників, T = %.0f тижнів%n", TIME);
        System.out.printf("Покупців %d: продано %d, продаж не відбувся %d, очікують %d — "
                + "баланс %s%n", r.customers, r.sold, r.lost, r.backlog,
                r.customers == r.sold + r.lost + r.backlog ? "виконується" : "НЕ виконується");
        System.out.printf("Середня кількість холодильників у запасі: %.4f%n", r.stock);
        System.out.printf("Середній час між продажами, що не відбулися: %.4f тижня%n", r.between);
        System.out.printf("Частка втрачених продажів: %.4f; замовлень %d, середній розмір %.2f%n",
                r.lostShare, r.orders, r.orderSize);


        System.out.println("\nВерифікація: зміна параметрів стратегії управління запасами");
        System.out.println(" рівень | точка | перегляд | запас  | втрачено | між втратами "
                + "| замовлень | розмір");
        int[][] runs = {{72, 18, 4}, {90, 18, 4}, {60, 18, 4}, {72, 30, 4}, {72, 10, 4},
            {72, 18, 2}, {72, 18, 8}};
        for (int[] p : runs) {
            Result v = run(p[0], p[1], p[2]);
            System.out.printf(" %6d | %5d | %8d | %6.2f | %8.4f | %11.3f | %9d | %6.2f%n",
                    p[0], p[1], p[2], v.stock, v.lostShare, v.between, v.orders, v.orderSize);
        }
    }
}
```

Результат роботи

```
Запаси холодильників, T = 20000 тижнів
Покупців 100109: продано 91614, продаж не відбувся 8495, очікують 0 — баланс виконується
Середня кількість холодильників у запасі: 28.8015
Середній час між продажами, що не відбулися: 2.3503 тижня
Частка втрачених продажів: 0.0849; замовлень 1446, середній розмір 63.33


Верифікація: зміна параметрів стратегії управління запасами
 рівень | точка | перегляд | запас  | втрачено | між втратами | замовлень | розмір
     72 |    18 |        4 |  28.73 |   0.0879 |        2.272 |      1436 |  63.57
     90 |    18 |        4 |  37.70 |   0.0693 |        2.880 |      1140 |  81.69
     60 |    18 |        4 |  22.89 |   0.1083 |        1.848 |      1711 |  52.00
     72 |    30 |        4 |  31.52 |   0.0221 |        9.013 |      1867 |  52.39
     72 |    10 |        4 |  27.70 |   0.1481 |        1.346 |      1223 |  69.80
     72 |    18 |        2 |  29.28 |   0.0417 |        4.784 |      1625 |  58.90
     72 |    18 |        8 |  27.57 |   0.1466 |        1.363 |      1239 |  68.88
```

Таблиця 3 – Верифікація моделі запасів: зміна параметрів стратегії

Рівень	Точка замовлення	Перегляд, тиж.	Середній запас	Частка втрачених продажів	Час між втратами, тиж.	Замовлень	Розмір замовлення
72	18	4	28,73	0,0879	2,27	1436	63,6
90	18	4	37,70	0,0693	2,88	1140	81,7
60	18	4	22,89	0,1083	1,85	1711	52,0
72	30	4	31,52	0,0221	9,01	1867	52,4
72	10	4	27,70	0,1481	1,35	1223	69,8
72	18	2	29,28	0,0417	4,78	1625	58,9
72	18	8	27,57	0,1466	1,36	1239	68,9

Висновок

За 20000 тижнів надійшло 100109 покупців, з них 91614 купили холодильник, 8495 пішли в інший магазин; баланс виконується. Середня кількість холодильників у запасі — 28,80, середній час між продажами, що не відбулися, — 2,35 тижня.

Верифікація (таблиця 3) показує, що модель реагує на зміну параметрів так, як і має реагувати система. Підвищення контрольного рівня збільшує середній запас і розмір замовлення та зменшує втрати; підвищення точки замовлення робить замовлення частішими і різко зменшує втрати; частіший перегляд запасу (2 тижні замість 4) удвічі зменшує частку втрачених продажів, а рідший (8 тижнів) — збільшує.

Завдання 3

Формалізація

Запит обробляється так: він потрапляє на головний сервер з імовірністю 0,7 або на допоміжний з імовірністю 0,3; якщо в черзі сервера вже 1000 запитів, запит відхиляється. Інакше він чекає вільного потоку; потік виконує обчислення, для якого потрібне ядро процесора, потім записує результат у спільну базу даних через одне з її з’єднань і тільки після цього повертає відповідь. Потік зайнятий від початку обробки до кінця запису, ядро — лише на час обчислення, з’єднання з БД — лише на час запису. Параметри, які в умові потрібно задати самостійно, наведено в таблиці 4; час — у мілісекундах.

Таблиця 4 – Параметри моделі серверів

Параметр	Головний сервер	Допоміжний сервер
Частка запитів	0,7	0,3
Кількість потоків	16	8
Кількість ядер процесора	8	4
Час обчислення, мс	експоненційний, 80	експоненційний, 100
Обмеження черги	1000	1000
Спільна база даних	10 з’єднань, запис — експоненційний, 5 мс	
Інтервал між запитами	експоненційний, 10 мс (100 запитів/с)	

Мережу наведено на рисунку 3. Вибір сервера — конфліктні переходи «на головний сервер» і «на допоміжний сервер» з імовірностями 0,7 і 0,3. Обмеження черги подано позицією «вільні місця в черзі» з 1000 маркерами: перехід «у чергу» (пріоритет 1) забирає вільне місце, а «відмова» (пріоритет 0) спрацьовує лише тоді, коли місць немає. Ресурси — потоки, ядра і з’єднання з БД — подано позиціями з відповідною кількістю маркерів, які забираються на початку дії і повертаються в кінці.

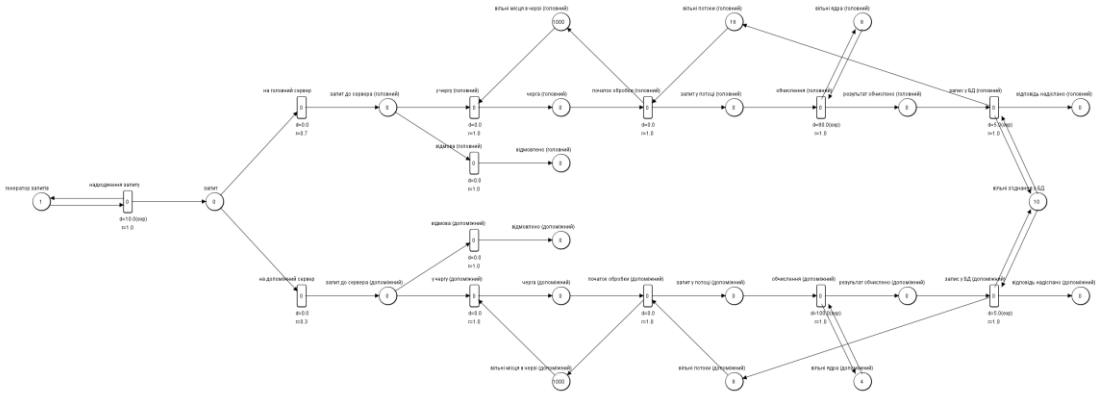


Рисунок 3 – Мережа Петрі обробки запитів на двох серверах зі спільною базою даних

Лістинг файлів

Файл src\Task3Servers.java:

```
/** Завдання 3: багатопотокова обробка запитів на головному і допоміжному серверах (мс). */
public class Task3Servers {
    static final double TIME = 600_000; // 10 хвилин
    static final String[][] SERVERS = {{"головний", "16", "8"}, {"допоміжний", "8", "4"}};


    record Server(int received, int rejected, int done, double queue, double threads,
                  double cores, double inSystem, double time) {
    }


    static Server server(Sim s, String[] srv) {
        String l = " (" + srv[0] + ")";
        int threads = Integer.parseInt(srv[1]), cores = Integer.parseInt(srv[2]);
        // запити на сервері: у черзі, у потоці, на обчисленні, з результатом і на записі в БД
        double inSystem = s.mean("черга" + l) + s.mean("запит у потоці" + l)
                + s.mean("результат обчислено" + l) + s.busy("обчислення" + l)
                + s.busy("запис у БД" + l);
        int done = s.mark("відповідь надіслано" + l);
        return new Server(s.fired("на " + srv[0] + " сервер"), s.mark("відмовлено" + l), done,
                s.mean("черга" + l), threads - s.mean("вільні потоки" + l),
                cores - s.mean("вільні ядра" + l), inSystem, inSystem / (done / TIME));
    }


    static Sim run(double interval) throws Exception {
        return new Sim(Nets.CreateNetServers(interval))
                .track("на головний сервер", "на допоміжний сервер").run(TIME);
    }


    public static void main(String[] args) throws Exception {
        Sim s = run(10);
        System.out.printf("Сервери, T = %.0f мс, запит у середньому кожні 10 мс%n", TIME);
        double inSystem = 0;
        int done = 0;
        for (String[] srv : SERVERS) {
            Server r = server(s, srv);
            inSystem += r.inSystem;
            done += r.done;
            System.out.printf("Сервер %s: надійшло %d, відмовлено %d, оброблено %d%n", srv[0],
                    r.received, r.rejected, r.done);
            System.out.printf("  середня черга %.3f; зайнято потоків %.3f з %s, ядер %.3f з %s%n",
                    r.queue, r.threads, srv[1], r.cores, srv[2]);
            System.out.printf("  середній час обробки запиту: %.2f мс%n", r.time);
        }
        System.out.printf("Зайнято з'єднань з БД: %.3f з 10%n",
                10 - s.mean("вільні з'єднання з БД"));
        System.out.printf("Середній час обробки запиту для обох серверів: %.2f мс%n",
                inSystem / (done / TIME));


        System.out.println("\nЗалежність часу обробки від інтенсивності запитів:");
        System.out.println(" інтервал, мс | запитів/с | ядер головного | ядер допоміжного "
```

```
                + "| час: головний | допоміжний | відмов");
        for (double interval : new double[]{40, 20, 12, 10, 9, 8}) {
            Sim v = run(interval);
            Server a = server(v, SERVERS[0]), b = server(v, SERVERS[1]);
            System.out.printf(" %12.0f | %9.0f | %14.3f | %16.3f | %13.1f | %10.1f | %6d%n",
                    interval, 1000 / interval, a.cores, b.cores, a.time, b.time,
                    a.rejected + b.rejected);
        }
    }
}
```

Результат роботи

```
Сервери, T = 600000 мс, запит у середньому кожні 10 мс
Сервер головний: надійшло 42273, відмовлено 0, оброблено 42267
  середня черга 0.035; зайнято потоків 6.674 з 16, ядер 5.645 з 8
  середній час обробки запиту: 95.24 мс
Сервер допоміжний: надійшло 17870, відмовлено 0, оброблено 17856
  середня черга 0.624; зайнято потоків 4.205 з 8, ядер 3.013 з 4
  середній час обробки запиту: 162.29 мс
Зайнято з'єднань з БД: 0.501 з 10
Середній час обробки запиту для обох серверів: 115.15 мс


Залежність часу обробки від інтенсивності запитів:
 інтервал, мс | запитів/с | ядер головного | ядер допоміжного | час: головний | допоміжний | відмов
           40 |        25 |          1.382 |            0.758 |          85.4 |       105.3 |      0
           20 |        50 |          2.787 |            1.509 |          84.2 |       108.5 |      0
           12 |        83 |          4.605 |            2.487 |          87.1 |       125.6 |      0
           10 |       100 |          5.627 |            2.990 |          93.8 |       154.9 |      0
            9 |       111 |          6.271 |            3.350 |         104.5 |       221.8 |      0
            8 |       125 |          7.097 |            3.839 |         140.5 |       785.4 |      0
```

Таблиця 5 – Залежність часу обробки запиту від інтенсивності запитів

Інтервал, мс	**Запитів/с**	**Зайнято ядер: головний**	**Допоміжний**	**Час обробки, мс: головний**	**Допоміжний**	**Відмов**
40	25	1,38	0,76	85,4	105,3	0
20	50	2,79	1,51	84,2	108,5	0
12	83	4,61	2,49	87,1	125,6	0
10	100	5,63	2,99	93,8	154,9	0
9	111	6,27	3,35	104,5	221,8	0
8	125	7,10	3,84	140,5	785,4	0

Висновок

При 100 запитах на секунду середній час обробки запиту становить 95,2 мс на головному сервері і 162,3 мс на допоміжному, у середньому 115,2 мс. Ядра головного сервера зайняті на 71 %, допоміжного — на 75 %, тому на допоміжному запити помітно частіше чекають ядра, і час обробки там більший, ніж час самого обчислення. Потоків вистачає з запасом (6,7 з 16 і 4,2 з 8), база даних майже вільна (0,50 з 10 з’єднань), відмов немає.

Модель коректна: при малій інтенсивності запитів час обробки наближається до суми середніх часів обчислення і запису — 85 мс для головного і 105 мс для допоміжного сервера (таблиця 5), а кількість зайнятих ядер дорівнює добутку інтенсивності на середній час обчислення (наприклад, 70 запитів/с · 0,08 с = 5,6 ядра головного сервера). Зі збільшенням інтенсивності час обробки зростає нелінійно; першим перевантажується допоміжний сервер, бо в нього менше ядер на одиницю навантаження. Для зменшення часу обробки доцільно додати ядра саме допоміжному серверу або змінити розподіл запитів між серверами.

Завдання 4

Для кожної мережі Петрі в каталозі nets репозиторію створено пакет із чотирьох файлів (таблиця 6). Зображення отримано з редактора (експорт зображення мережі), PNML-файл — командою збереження редактора, Java-метод — командою Save net as method.

Таблиця 6 – Пакет файлів мереж Петрі

Каталог	Мережа	Файли
nets/1_conveyor	Конвеєр (завдання 1 практикуму 5)	task.txt, net.png, net.pnml, method.txt
nets/2_fridges	Запаси холодильників (завдання 4 практикуму 5)	task.txt, net.png, net.pnml, method.txt
nets/3_servers	Сервери зі спільною БД (завдання 3)	task.txt, net.png, net.pnml, method.txt

Файл task.txt містить текст завдання (для мережі серверів — разом із самостійно заданими параметрами), net.png — зображення мережі, net.pnml — мережу у форматі PNML (ISO/IEC 15909-2), яку можна відкрити в редакторі, method.txt — Java-метод, що створює мережу.

Завдання 5

Стохастична мережа Петрі задається як N = (P, T, A, W, I, K, R), де P — множина позицій, T — множина переходів, A — множина дуг, W — кратності дуг, I — інформаційні дуги, K = {(c_T, b_T)} — пріоритети і ймовірності запуску переходів, R — часові затримки. Стан мережі в момент t описується маркіруванням M(P, t) і, для кожного переходу, множиною моментів виходу маркерів E_T(t); кількість маркерів у переході B_T(t) = |E_T(t)|.

Функціонування мережі описується такими рівняннями.

Умова запуску: Z_T(t) = 1, якщо M(P, t) ≥ W(P, T) для всіх P ∈ •T (разом з інформаційними дугами), інакше Z_T(t) = 0.

Розв'язання конфлікту: з переходів, для яких Z_T(t) = 1, вибирається перехід з найбільшим пріоритетом c_T; серед рівних — з імовірністю b_T.

Вхід маркерів у вибраний перехід T: M(P, t+0) = M(P, t) − W(P, T) для P ∈ •T, крім інформаційних дуг; E_T := E_T ∪ {t + τ_T}, де τ_T — значення затримки, згенероване за законом R(T).

Вихід маркерів у момент t = min E_T: M(P, t+0) = M(P, t) + W(T, P) для P ∈ T•; E_T := E_T \ {t}.

Просування часу: t_next = min по T ∈ T з min E_T(t).

Для мережі серверів позначимо s ∈ {г, д} — головний і допоміжний сервер, n_г = 16, n_д = 8 — кількості потоків, c_г = 8, c_д = 4 — кількості ядер, μ_г = 80 мс, μ_д = 100 мс — середні часи обчислення, p_г = 0,7, p_д = 0,3. Позиції: P0 — генератор запитів, P1 — запит, Ps2 — запит до сервера s, Ps3 — вільні місця в черзі s, Ps4 — черга s, Ps5 — відмовлено s, Ps6 — вільні потоки s, Ps7 — запит у потоці s, Ps8 — вільні ядра s, Ps9 — результат обчислено s, Ps10 — відповідь надіслано s, РБД — вільні з'єднання з БД. Початкове маркірування: M(P0) = 1, M(Ps3) = 1000, M(Ps6) = n_s, M(Ps8) = c_s, M(РБД) = 10, решта — 0. Рівняння переходів наведено в таблиці 7.

Таблиця 7 – Рівняння переходів мережі серверів

Перехід	Умова запуску	Вхід маркерів	Вихід маркерів	Затримка; пріоритет
T0 надходження запиту	M(P0) ≥ 1	M(P0) − 1	M(P0) + 1, M(P1) + 1	Exp(10); 0
Ts1 на сервер s	M(P1) ≥ 1	M(P1) − 1	M(Ps2) + 1	0; 1, ймовірність p_s
Ts2 у чергу	M(Ps2) ≥ 1, M(Ps3) ≥ 1	M(Ps2) − 1, M(Ps3) − 1	M(Ps4) + 1	0; 1
Ts3 відмова	M(Ps2) ≥ 1	M(Ps2) − 1	M(Ps5) + 1	0; 0
Ts4 початок обробки	M(Ps4) ≥ 1, M(Ps6) ≥ 1	M(Ps4) − 1, M(Ps6) − 1	M(Ps3) + 1, M(Ps7) + 1	0; 0
Ts5 обчислення	M(Ps7) ≥ 1, M(Ps8) ≥ 1	M(Ps7) − 1, M(Ps8) − 1	M(Ps8) + 1, M(Ps9) + 1	Exp(μ_s); 0
Ts6 запис у БД	M(Ps9) ≥ 1, M(РБД) ≥ 1	M(Ps9) − 1, M(РБД) − 1	M(РБД) + 1, M(Ps6) + 1, M(Ps10) + 1	Exp(5); 0

Оскільки переходи Ts2 і Ts3 конфліктують за маркер позиції Ps2, а пріоритет Ts2 вищий, відмова відбувається лише тоді, коли M(Ps3) = 0, тобто черга заповнена. З рівнянь випливають інваріанти мережі — величини, що зберігаються протягом усього моделювання:

M(Ps3) + M(Ps4) = 1000 — місця в черзі s вільні або зайняті;

M(Ps6) + M(Ps7) + B(Ts5) + M(Ps9) + B(Ts6) = n_s — кожен потік вільний або обробляє запит;

M(Ps8) + B(Ts5) = c_s — кожне ядро вільне або виконує обчислення;

M(РБД) + B(Tг6) + B(Tд6) = 10 — кожне з'єднання з БД вільне або виконує запис.

Шукана характеристика — середній час обробки запиту на сервері s — визначається за формулою Літтла:

$$W_s = L_s / \lambda_s,\quad L_s = \bar{M}(Ps4) + \bar{M}(Ps7) + \bar{M}(Ps9) + \bar{B}(Ts5) + \bar{B}(Ts6),\quad \lambda_s = M(Ps10, T_{mod}) / T_{mod},$$

де риска означає середнє за часом значення, T_mod — час моделювання. Саме так середній час обробки обчислює програма Task3Servers.

Висновок

У ході виконання комп’ютерного практикуму опановано бібліотеку PetriObjModelPaint: мережі Петрі побудовано в її графічному редакторі, збережено як Java-методи і реалізовано алгоритмом імітації класу PetriSim у моделі PetriObjModel.

1. Модель конвеєра верифіковано зміною інтенсивності надходження: середня кількість зайнятих пристроїв дорівнює λ·tобр, час перебування за формулою Літтла збігається з часом, підрахованим через переміщення; при 4 деталях/хв деталь перебуває в системі 9,9 хв.
2. Модель запасів холодильників верифіковано зміною контрольного рівня, точки замовлення і періоду перегляду; середній запас — 28,80, середній час між продажами, що не відбулися, — 2,35 тижня.
3. Побудовано модель багатопотокової обробки запитів на двох серверах зі спільною БД; середній час обробки запиту — 95 мс на головному і 162 мс на допоміжному сервері; «вузьке місце» — ядра допоміжного сервера.
4. Для всіх мереж підготовлено пакет файлів (текст завдання, зображення, PNML, Java-метод), а мережу серверів описано рівняннями умов запуску, входу і виходу маркерів та інваріантами.

Під час роботи з’ясовано, що бібліотека розв’язує конфлікт серед усіх активних переходів однакового пріоритету, а не лише серед тих, що мають спільні позиції; тому переходам вибору за ймовірністю потрібно задавати пріоритет, вищий за пріоритет переходів-генераторів.

Посилання

1. Комп’ютерний практикум 6 «Застосування алгоритму стохастичної мережі Петрі для реалізації моделей дискретно-подійних систем»: методичні вказівки з дисципліни «Комп’ютерне моделювання дискретно-подійних систем». – КПІ ім. Ігоря Сікорського, 2026.
2. Стеценко І. В. Моделювання систем: навч. посіб. – Черкаси: ЧДТУ, 2010. – 399 с.
3. PetriObjModelPaint — графічний редактор і симулятор мереж Петрі та Петрі-об’єктних моделей. – URL: https://github.com/StetsenkoInna/PetriObjModelPaint
4. Повний код практичної роботи: https://github.com/ip34is/7-sem/tree/main/CMDES/CMDES-lab6